

# Programming Fundamentals 2

---

## Topic 08 - Tutorial Sheet 8

---

### Inheritance, Subtyping and Polymorphism

---

## Part 1 – Motivation for Inheritance

---

Consider the following two classes in a blogging application.

```
public class BlogPost
{
    private String author;
    private String title;
    private int likes;
}
public class VideoPost
{
    private String author;
    private String videoURL;
    private int likes;
}
```

### Exercise 1

---

Identify the duplicate fields that appear in both classes.

Explain why duplication like this can cause problems in software systems.

Suggest how inheritance could help improve this design.

### Exercise 2

---

Suppose we create a superclass called Content.

Which fields should be placed in the Content class?

Which fields should remain in the subclasses?

Draw a simple class hierarchy.

Example structure:

```
Content
├─ BlogPost
└─ VideoPost
```

## Part 2 – Implementing Inheritance

---

Consider the following superclass.

```
public class Content
{
    private String author;
    private int likes;
}
```

### Exercise 3

---

Write the class header for the following subclasses so that they inherit from Content.

BlogPost VideoPost

### Exercise 4

---

Why might the following fields belong in the superclass?

```
author

likes

timestamp
```

Explain why these are common to all posts.

## Part 3 – Subtyping

---

Consider the following hierarchy.

```
Animal
├─ Dog
└─ Cat
```

### Exercise 5

---

Which of the following assignments are valid?

```
Animal a1 = new Animal();
Animal a2 = new Dog();
Animal a3 = new Cat();
```

Explain why they are valid or invalid.

## Exercise 6

---

Explain the following concept in your own words.

A subclass object can be used anywhere a superclass object is expected.

Give an example using the Animal hierarchy.

## Part 4 – Subtyping and Method Calls

---

Consider the following method.

```
public void addContent(Content item)
```

## Exercise 7

---

Which of the following method calls are valid?

```
BlogPost b = new BlogPost(...);
addContent(b);
VideoPost v = new VideoPost(...);
addContent(v);
addContent(new Animal());
```

Explain your answers.

## Part 5 – Polymorphic Variables

---

Consider the following declaration.

```
Content c;
```

## Exercise 8

---

What objects could be stored in this variable?

Give two examples.

## Exercise 9

---

Explain the difference between:

```
Content c = new Content();
```

and

```
Content c = new BlogPost();
```

## Part 6 – Casting

---

Consider the following code.

```
Animal a;  
Dog d = new Dog();  
  
a = d;
```

## Exercise 10

---

Is the following assignment valid?

```
d = a;
```

Explain why or why not.

## Exercise 11

---

Explain the purpose of casting in this example:

```
d = (Dog) a;
```

What might happen if a was actually referring to a Cat object?

## Part 7 – The Object Class

---

## Exercise 12

---

What is the root superclass of all classes in Java?

Why is this useful when designing collections or APIs?

## Part 8 – Polymorphic Collections

---

Consider the following collection.

```
ArrayList<Content> items;
```

### Exercise 13

---

What types of objects could be stored in this list?

Give two examples.

### Exercise 14

---

Explain why the following declaration might require casting.

```
ArrayList<Object> items;
```

What problem might arise when retrieving elements from this list?

## Part 9 – Wrapper Classes

---

Primitive types cannot be stored directly in collections.

### Exercise 15

---

Complete the following table.

| Primitive Type | Wrapper Class |
|----------------|---------------|
| int            | ?             |
| double         | ?             |
| boolean        | ?             |
| char           | ?             |

### Exercise 16

---

Explain the difference between the following two pieces of code.

```
int value = 20;
```

and

```
Integer wrapper = new Integer(value);
```

What is happening in the above code?

## Part 10 – Autoboxing and Unboxing

---

Consider the following code.

```
ArrayList<Integer> scores = new ArrayList<>();  
  
scores.add(90);  
  
int firstScore = scores.get(0);
```

### Exercise 17

---

What is autoboxing?

What is unboxing?

Identify where they occur in the code above.

### Challenge Question

---

Suppose we introduce a new subclass:

```
AudioPost extends Content
```

### Exercise 18

---

Explain why no changes are needed to this method:

```
public void addContent(Content item)
```

Why is this an advantage of inheritance?

---